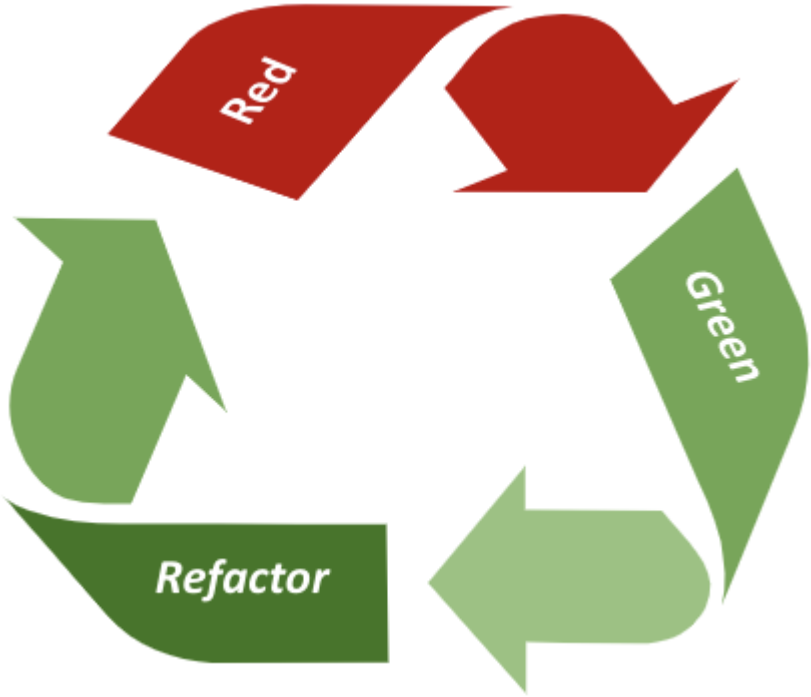
Ejercicio Guiado 3
Enunciado



Refactoring y Diseño Simple

Desarrollo de Software

Abril 2026

Grado en Ingeniería Informática - Doble Grado en Informática y ADE

Tabla de contenido

1. OBJETIVOS Y EXPOSICIÓN DE CASO *3*

2. REFACTORING *4*

Paso 2.1: Identificación y resolución de las situaciones de refactoring. **4**

Paso 2.2: Actualización del código según la normativa de código. **6**

Paso 2.3: Ejecución de los casos de prueba definidos. **6**

Paso 2.4: Registro del código funcional y de pruebas en GitHub **6**

3. DISEÑO SIMPLE *7*

Paso 3.1: Aplicación del patrón Singleton para asegurar que solo se puede generar una instancia de una clase **7**

Paso 3.2: Registro del código funcional y de pruebas en GitHub **9**

4. Publicaciones relacionadas con el refactoring. *9*

5. TAREAS A REALIZAR *10*

6. NORMAS Y PROCEDIMIENTOS *11*

1. OBJETIVOS Y EXPOSICIÓN DE CASO

Este ejercicio guiado tiene como objetivos:

- Poner en práctica la técnica de refactoring.
- Poner en práctica los principios de diseño simple y patrones de diseño básico.

Como punto de partida de este ejercicio guiado se proporcionará el código fuente que implementa satisfactoriamente algunas funcionalidades relacionadas con las presentadas en el ejercicio guiado anterior, por tanto, como primer paso, será necesario asegurar que los casos de prueba definidos permiten verificar satisfactoriamente el código fuente. Es muy importante tener en cuenta que el código proporcionado implementa algunas funcionalidades descritas en el enunciado del ejercicio guiado 2 sin adaptaciones que se hayan podido dar durante las clases de prácticas. También se añade una función nueva que recupera los documentos para una fecha determinada. Esta función admite cualquier fecha con formato válido y retorna el número de documentos encontrados, así como un registro en un fichero de log. Se incluyen las pruebas básicas para comprobar esta funcionalidad durante el proceso de refactorización.

En primer lugar, se deberá reorganizar el código de tal manera que se puedan resolver distintas situaciones predefinidas de refactoring que se han introducido intencionadamente en el código para facilitar el aprendizaje de esta técnica. Los casos propuestos para la aplicación de esta técnica se proporcionan en la sección 2.

Posteriormente, se abordarán diversas mejoras de la arquitectura técnica propuesta para el código proporcionado que permiten aplicar los principios fundamentales de diseño simple y algunos patrones de diseño básicos. Las recomendaciones de aplicación de patrones se describen en el capítulo 3 de este enunciado.

Esta práctica no tiene un documento de memoria: por tante **es obligatorio hacer commit y push después de cada paso del proceso de refactorización, incluyendo un comentario significativo de los cambios realizados**.

2. REFACTORING

Se han identificado las siguientes situaciones susceptibles de reorganización en el código fuente del componente que está desarrollando la compañía UC3M Consulting. Las acciones de reorganización de código se deberán realizar en el orden propuesto a continuación y al finalizar cada etapa se deberá comprobar que las pruebas siguen funcionando correctamente. En caso de necesitar actualizar algún caso de prueba, deberá consultar antes con el profesor de prácticas.

Paso 2.1: Identificación y resolución de las situaciones de refactoring.

Las situaciones de refactoring que están incluidas intencionadamente en el código son las siguientes:

a) Nombres misteriosos. Escribir texto desconcertante es apropiado cuando se está escribiendo o leyendo una novela de detectives, pero no cuando estás leyendo un código. Una de las partes más importantes del código claro son los buenos nombres, por lo que pensamos mucho en las funciones de nomenclatura, módulos, variables, clases, para que comuniquen claramente lo que hacen y cómo usarlos. Las refactorizaciones más comunes en este caso son los renombrados: cambiar la declaración de la función (para renombrar una función), Renombrar variable y Renombrar campo.
b) Si ve la misma estructura de código en más de un lugar, puede estar seguro de que su programa será mejor si encuentra la manera de unificarlos. La duplicación significa que cada vez que lea estas copias, deberá leerlas cuidadosamente para ver si hay alguna diferencia. Si necesita cambiar el código duplicado, debe encontrar y capturar cada duplicación. El problema de código duplicado más simple es cuando tiene la misma expresión en dos métodos de la misma clase. Entonces todo lo que tiene que hacer es Extraer Función e invocar el código desde ambos lugares. Si tiene un código similar, pero no del todo idéntico, vea si puede usar Slide Statements para organizar el código de modo que los elementos similares estén todos juntos para una extracción fácil. Si los fragmentos duplicados están en subclases de una superclase común, puede usar el Método

Pull Up para evitar llamar a uno de otro.

c) Funciones Largas. En nuestra experiencia, los programas que duran más tiempo y tienen menos problemas son aquellos que tienen funciones o métodos cortos. Desde los primeros días de la programación, los programadores se han dado cuenta de que cuanto más larga es una función, más difícil es entenderla. El noventa y nueve por ciento de las veces, todo lo que tiene que hacer para acortar una función es Extraer Función. Encuentra partes de la función que puedan ir bien juntas y crea una nueva.

d) Cambios Divergentes. Estructuramos nuestro software para facilitar el cambio; después de todo, el software está destinado a ser adaptable sin problemas. Cuando hacemos un cambio, queremos poder saltar a un punto claro en el sistema y hacer el cambio. El cambio divergente ocurre cuando un módulo se cambia a menudo de diferentes maneras por diferentes razones. Si observa un módulo y dice: “Bueno, tendré que cambiar estas tres funciones cada vez que cambie el nombre de un fichero; Tengo que cambiar estas cuatro funciones cada vez que aparezca un nuevo tipo de cambio financiero”, esto es una indicación de un cambio divergente. Si los dos aspectos forman una secuencia, por ejemplo, primero se obtienen datos de la base de datos y luego aplica el procesamiento financiero, entonces se aplica Split Phase separa los dos con una clara estructura de datos entre ellos. Si las funciones combinan los dos tipos de procesamiento, utilice Extraer Función para separarlas antes de moverlas. Si los módulos son clases, entonces Extraer Clase ayuda a formalizar cómo hacer la división.

e) Clases grandes. Cuando una clase está tratando de hacer demasiado, a menudo aparece con demasiados campos y métodos. Cuando una clase tiene demasiados campos, el código duplicado no puede estar muy lejos. Puede Extraer la Clase para agrupar un número de las variables. Elija variables para ir juntas en la nueva clase que tengan sentido. Si en la situación la herencia tiene sentido, Extraer Superclase o Reemplazar código de tipo con subclases (que esencialmente es extraer una subclase) a menudo son más fáciles.

f) Comentarios. La razón por la que mencionamos los comentarios aquí es porque con frecuencia ocurre que se encuentra código con comentarios abundantes y te

das cuenta de que los comentarios están ahí porque el código es malo. Los comentarios nos llevan a un código erróneo que tiene todos los defectos negativos que deben conducir a refactoring. Cuando terminamos, a menudo encontramos que los comentarios son superfluos. Si necesita un comentario para explicar lo que hace un bloque de código, intente Extraer función. Si el método ya está extraído pero aún necesita un comentario para explicar lo que hace, use Cambiar declaración de función para cambiar el nombre.

Paso 2.2: Actualización del código según la normativa de código.

Por último, se deberán realizar las actualizaciones correspondientes para que el código fuente incluya convenientemente las reglas y recomendaciones definidas en la normativa de código PEP8 validadas a través de pyLint. El código proporcionado se ha limpiado, pero contiene aún algunos errores que deben solucionarse durante el proceso de refactorización.

Paso 2.3: Ejecución de los casos de prueba definidos.

Finalmente se debe verificar que el resultado del refactoring es satisfactorio y cumple las expectativas marcadas. Con este propósito se deberán volver a ejecutar las pruebas creadas para verificar el correcto funcionamiento del componente EnterpriseManager.

Es necesario considerar que este paso se debe ejecutar inmediatamente después de la finalización de cada una de las tareas de reorganización de código. Por ejemplo, si se extrae un método, hay que ejecutar las pruebas inmediatamente después para comprobar que el comportamiento del programa no ha cambiado.

En caso de necesitar actualizar algún caso de prueba como resultado del proceso de refactoring, deberá consultar antes con el profesor de prácticas.

Paso 2.4: Registro del código funcional y de pruebas en GitHub

Es obligatorio que, tras cada uno de los cambios, una vez comprobado que las pruebas siguen ejecutando correctamente, realice un commit incluyendo en los comentarios una pequeña descripción del cambio realizado. Esto será la documentación de su ejercicio, puesto que el profesor podrá ver paso a paso los cambios

realizados.

Como la resolución del ejercicio guiado supondrá diversas sesiones de trabajo, cada sesión debe finalizar con la correcta ejecución de los casos de prueba y el envío de código correspondiente.

No están permitidos los envíos de código en los que se hayan realizado gran cantidad de cambios. Un caso extremo sería un repositorio en el que el primer envío es el código original y que solo haya un commit tras el cuál todo el código está actualizado.

3. DISEÑO SIMPLE

En esta sección se abordarán diversas mejoras de la arquitectura técnica propuesta para la solución del ejercicio guiado 2 que permiten aplicar los principios fundamentales de diseño simple y algunos patrones de diseño básicos.

Solamente se podrá ejecutar una instancia de la clase EnterpriseManager y de las clases que se creen para gestionar cada fichero de datos. Para ello se abordará la aplicación del patrón Singleton.

Paso 3.1: Aplicación del patrón Singleton para asegurar que solo se puede generar una instancia de una clase

En ingeniería de software, singleton o instancia única es un patrón de diseño que permite restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto.

Su intención consiste en garantizar que una clase solo tenga una instancia y proporcionar un punto de acceso global a ella.

El patrón singleton se implementa creando en nuestra clase un método que crea una instancia del objeto solo si todavía no existe alguna. Para asegurar que la clase no puede ser instanciada nuevamente se regula el alcance del constructor (con modificadores de acceso como protegido o privado).

A continuación, se muestra un ejemplo del código en Python que es necesario para

implementar el patrón singleton [1]:

```
class SoyUnico(object):

     class __SoyUnico:

       def __init__(self):

            self.nombre = None

       def __str__(self):

             return `self` + ' ' + self.nombre

      instance = None



    def __new__(cls):

       if not SoyUnico.instance:

            SoyUnico.instance = SoyUnico.__SoyUnico()

       return SoyUnico.instance



    def __getattr__(self, nombre):

       return getattr(self.instance, nombre)



    def __setattr__(self, nombre, valor):

       return setattr(self.instance, nombre, valor)
```

[1] Ejemplo obtenido de https://jarroba.com/patron-singleton-python-ejemplos/

Paso 3.2: Registro del código funcional y de pruebas en GitHub

Es obligatorio que, tras cada uno de los cambios, una vez comprobado el correcto funcionamiento con la ejecución de las pruebas, se envíe el código al repositorio. Por ejemplo, tras aplicar el patrón singleton a una clase y comprobar que las pruebas se ejecutan correctamente, se enviará el código al repositorio. En cada commit incluye un comentario describiendo los cambios realizados: esto será la documentación de tu ejercicio.

4. Publicaciones relacionadas con el refactoring.

Más allá de la aplicación práctica, se está investigando en la aplicación de las técnicas de Refactoring. Por ejemplo, en el artículo: "Energy Efficiency Analysis of Code Refactoring Techniques for Green and Sustainable Software in Portable Devices" (https://www.mdpi.com/2079-9292/11/3/442) se analiza el impacto del refactoring en la eficiencia energética.

Realiza una búsqueda en Google Scholar (https://scholar.google.com) sobre publicaciones relacionadas con refactoring. En un único documento de word, **cada miembro** del equipo incluirá la referencia (título, nombre de la revista o congreso donde se ha publicado, dirección web donde localizarlo, autores y fecha de publicación) a **dos artículos** que se hayan publicado en los últimos dos años. Cada alumno incluirá un **breve resumen** de los artículos (**un párrafo** por cada uno) explicando brevemente por qué los considera relevantes.

5. TAREAS A REALIZAR

1. **Cree un nuevo repositorio en GitHub para este ejercicio guiado**. Nombra este repositorio de acuerdo con las reglas descritas en el ejercicio guiado 2 (**GXX.2026.TYY.EG3**) e **invita a tu profesor de prácticas**.
2. Clona el proyecto e incluye el código y la estructura de carpetas disponibles en Aula Global.
3. Aplicar las técnicas de refactorización y diseño necesarios, tanto los mencionados como los que se identifiquen en las diferentes sesiones de prácticas.
4. **Es obligatorio subir el código refactorizado y rediseñado tras cada paso en el proceso**. Es obligatorio registrar cada uno de los cambios con un commit, de forma que **el historial de envíos muestre paso a paso el proceso que se ha seguido**. Esta será la documentación de vuestro proyecto**. No se puede aprobar si no se cumple este apartado**.
5. El código debe cumplir con el estándar PEP8 de acuerdo con la herramienta pyLint.
6. Realizar la búsqueda de los artículos, e incluir el documento con la referencia y el resumen de los artículos en una carpeta llamada “doc”.
7. Incluir en el envío final de la práctica, el último informe en XML generado por pyBuilder con la ejecución de todos los casos de prueba. Puede ejecutar pyb clean antes de ejecutar pyb por última vez para localizar fácilmente el último informe. Guarde este informe en la carpeta “doc”.
8. Desde github, descargar el fichero zip con el contenido completo del proyecto y subir el zip en el entregador de Aula Global habilitado por el profesor de prácticas.

6. NORMAS Y PROCEDIMIENTOS

Este ejercicio se completará en parejas. Si algún alumno no realiza ningún envío o se detecta que el número de envíos con cambios significativos es muy bajo en comparación con su compañero puede recibir la calificación de 0 puntos. Entendemos por cambios significativos, aquellos que están relacionados con el proceso de refactorización propuesto. Un ejemplo de cambio no significativo sería un commit en el que se ha añadido un espacio dentro de un comentario y no se ha cambiado ningún otro elemento del código. **De igual forma, no realizar commit y push con comentarios tras cada cambio puede suponer una calificación de 0 puntos.**

La fecha límite para este ejercicio es el 24 de abril de 2026 antes de las 23:59.

De acuerdo con los estándares de evaluación continua establecidos en esta asignatura, si un equipo no publica la solución del ejercicio antes de la fecha límite, el ejercicio será evaluado con una puntuación de 0 puntos.